

**“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA
DEMOCRACIA”**

**SISTEMA DE GESTIÓN DE PROCESOS
DE WINDOWS IMPLEMENTADO
MEDIANTE ESTRUCTURAS
DINÁMICAS LINEALES EN DEV C++**

Docente: Rosario Delia Osorio Contreras

INTEGRANTES:

Flores Torres Elvis Augusto

Carrión Herquinio Kheynell Alessander

Laura Torres Diego Fernando



**Universidad
Continental**

HUANCAYO - JUNÍN - PERÚ

2026

1. Análisis del Problema

1.1. Descripción del Contexto

Los sistemas operativos modernos, como Windows 10/11, ejecutan decenas de procesos de forma simultánea. Por ejemplo, al abrir Google Chrome, Spotify, Discord y Visual Studio Code al mismo tiempo, el sistema debe administrar la memoria RAM, el tiempo de CPU y las prioridades de cada proceso. Sin una gestión correcta, el sistema puede volverse lento, bloquearse o perder rendimiento.

El presente proyecto tiene como finalidad simular, mediante estructuras de datos dinámicas lineales implementadas desde cero en C++ (DEV C++), el comportamiento básico de un administrador de tareas de Windows, abarcando el registro de procesos, la planificación del CPU y la gestión de memoria.

1.2. Identificación del Problema

Windows ejecuta múltiples aplicaciones simultáneamente y necesita:

- Mantener un registro actualizado de todos los procesos activos.
- Decidir qué proceso debe ejecutarse primero según su prioridad.
- Asignar y liberar memoria RAM de forma eficiente cuando los procesos inician o terminan.
- Persistir el estado del sistema para recuperarlo ante un reinicio o cierre inesperado.

1.3. Procesos Simulados

ID	Nombre del Proceso	Descripción
1001	Google Chrome	Navegador web principal
1002	Spotify	Reproducción de música en streaming
1003	Discord	Comunicación por voz y texto
1004	Visual Studio Code	Editor de código fuente
1005	Explorador de Archivos	Gestión del sistema de archivos
1006	Windows Defender	Antivirus y protección del sistema
1007	Actualización de Windows	Descarga e instalación de parches
1008	Descargas en segundo plano	Gestión de transferencias de archivos
1009	Zoom / Google Meet	Videollamadas y conferencias

2. Requerimientos del Sistema

2.1. Requerimientos Funcionales

N°	Función	Descripción
RF01	Registrar procesos	Ingresar nuevos procesos con ID, nombre, prioridad, estado y memoria utilizada.

RF02	Buscar procesos	Localizar un proceso por su ID o por su nombre.
RF03	Modificar prioridad	Actualizar el nivel de prioridad de un proceso activo.
RF04	Eliminar procesos	Retirar un proceso de la lista de activos cuando finaliza.
RF05	Organizar por prioridad	Encolar procesos según su nivel de prioridad para simular la planificación del CPU.
RF06	Administrar memoria	Asignar (push) y liberar (pop) bloques de memoria usando una pila dinámica.
RF07	Persistencia de datos	Guardar el estado del sistema en archivos TXT y cargarlo al iniciar.
RF08	Visualizar estado	Mostrar la lista de procesos, la cola del planificador y el estado de la memoria.

2.2. Requerimientos No Funcionales

N°	Categoría	Descripción
RNF01	Plataforma	Desarrollado en C++ con DEV C++; ejecutable en Windows 10 o superior.
RNF02	Hardware mínimo	Procesador Intel Core i3 o equivalente; 4 GB de RAM mínimo.
RNF03	Rendimiento	Las operaciones básicas (insertar, buscar, eliminar) deben completarse en menos de 2 segundos.
RNF04	Gestión de memoria	Uso eficiente de memoria dinámica; sin fugas de memoria (memory leaks) durante la ejecución.
RNF05	Mantenibilidad	Código fuente modular, comentado y con estructura clara que facilite futuras mejoras.
RNF06	Interfaz	Interfaz por consola (CLI), sin necesidad de entorno gráfico avanzado.
RNF07	Almacenamiento	Persistencia mediante archivos TXT de tamaño reducido, sin bases de datos externas.

3. Diseño de la Solución

Módulo	Estructura de datos	Función principal
Gestor de Procesos	Lista enlazada simple	Registro y consulta de todos los procesos activos
Planificador de CPU	Cola de prioridad dinámica	Ordenar y ejecutar procesos según su prioridad
Gestor de Memoria	Pila dinámica (LIFO)	Asignación y liberación de bloques de RAM

4. Estructura de Datos Implementadas y Justificación

4.1. Lista Enlazada

La lista enlazada simple fue elegida para el módulo de gestión de procesos porque permite almacenar un número variable de procesos sin necesidad de reservar memoria fija de antemano. Cada nodo se crea dinámicamente al registrar un proceso y se libera al eliminarlo, lo que evita el desperdicio de memoria.

Esta estructura es adecuada para el gestor de procesos porque:

- La inserción al final de la lista es sencilla y no requiere reorganizar toda la estructura.
- La búsqueda por ID o nombre recorre la lista nodo a nodo, lo que es aceptable para el tamaño esperado del sistema (menos de 50 procesos simultáneos).
- La eliminación de un nodo en cualquier posición se realiza redirigiendo el puntero del nodo anterior, sin mover datos.
- Refleja la forma en que Windows mantiene su tabla de procesos: una colección dinámica que crece y decrece a medida que los procesos se inician y terminan.

4.2. Cola de Prioridad Dinámica

La cola de prioridad dinámica fue elegida para el planificador porque permite modelar el comportamiento real de los planificadores de CPU de Windows, donde los procesos no se ejecutan en orden de llegada sino según su importancia o urgencia.

Por ejemplo, Windows Defender (alta prioridad) debe ejecutarse antes que Spotify (baja prioridad), aunque Spotify haya sido registrado primero. La cola de prioridad garantiza este comportamiento al insertar siempre en la posición correcta según el nivel de prioridad.

Esta estructura es adecuada porque:

- El encolamiento mantiene el orden de prioridad automáticamente en cada inserción.
- El desencolamiento siempre extrae el proceso de mayor prioridad (el del frente), simulando la ejecución del CPU.
- Es dinámica: no tiene límite fijo de capacidad, a diferencia de un arreglo estático.

4.3. Pila Dinámica

La pila dinámica fue elegida para el gestor de memoria porque sigue el principio LIFO (Last In, First Out), que refleja cómo los bloques de memoria del stack de un proceso se asignan y liberan: el último bloque asignado es siempre el primero en liberarse cuando el proceso termina.

Adicionalmente, la pila permite mantener un historial de procesos cerrados recientemente: si el usuario cierra primero Chrome, luego Discord y finalmente Spotify, al consultar el historial aparecerá primero Spotify (el último cerrado), luego Discord y finalmente Chrome.

Esta estructura es adecuada porque:

- Las operaciones push y pop son de tiempo constante $O(1)$, lo que garantiza una respuesta inmediata.

- Modela correctamente la liberación de memoria tipo LIFO que utiliza el stack de llamadas de cualquier proceso.
- Permite un historial de acciones que puede usarse para "deshacer" operaciones recientes.

4.4. Justificación General de la Elección

La combinación de estas tres estructuras permite cubrir los tres aspectos fundamentales de la administración de procesos en un sistema operativo:

Necesidad del SO	Estructura elegida	Razón
Registro dinámico de procesos	Lista enlazada	Tamaño variable, inserción/eliminación eficiente en cualquier posición
Planificación por prioridad	Cola de prioridad	Garantiza que el proceso más urgente siempre se ejecute primero
Gestión de memoria RAM	Pila dinámica	LIFO refleja la asignación/liberación real del stack de procesos

5. Diseño de la Solución

5.1. Descripción general de la solución

El sistema propuesto consiste en una simulación básica de un gestor de procesos de Windows, desarrollado en C++ mediante Dev C++, utilizando estructuras de datos dinámicas lineales.

La finalidad del sistema es representar de manera simplificada cómo un sistema operativo administra procesos activos, organiza su ejecución según prioridad y simula el uso de memoria. Para ello, se utilizan procesos comunes de Windows, como Google Chrome, Spotify, Discord, Visual Studio Code, Explorador de Archivos, Windows Defender y Actualización de Windows.

La solución estará dividida en tres módulos principales:

Campo	Descripción
ID	Identificador único del proceso.
Nombre	Nombre del proceso o aplicación.
Prioridad	Nivel de importancia del proceso.
Estado	Situación actual del proceso: nuevo, listo, en ejecución, bloqueado o finalizado.
Memoria	Cantidad de memoria utilizada por el proceso.

Esta organización permite dividir el programa en módulos claros, evitando mezclar responsabilidades y facilitando la explicación del funcionamiento durante la exposición.

5.2. Descripción de estructuras de datos y operaciones

5.2.1 Lista Enlazada Simple

La lista enlazada simple se utilizará para almacenar todos los procesos registrados en el sistema. Cada proceso estará representado por un nodo que contendrá sus datos principales y un puntero hacia el siguiente nodo. Esta estructura es adecuada porque permite manejar una cantidad variable de procesos sin establecer un límite fijo desde el inicio.

Estructura del nodo:

```
struct NodoProceso {  
    int id; // Identificador único del proceso  
    string nombre; // Nombre del proceso (ej. "Chrome")  
    int prioridad; // Nivel 1 (baja) a 5 (alta)  
    string estado; // "activo" | "suspendido" | "terminado"  
    int memoria; // Megabytes de RAM asignados  
    NodoProceso* siguiente;  
};
```

Operación	Acción	Estructura de control usada
insertarProceso()	Agrega un nodo al final de la lista	WHILE (recorre hasta el último nodo) + IF-ELSE (lista vacía)
buscarPorID(id)	Recorre la lista hasta encontrar el ID	WHILE + IF
buscarPorNombre(n)	Recorre la lista buscando el nombre	WHILE + IF
modificarPrioridad(id, p)	Localiza el nodo y actualiza su prioridad	WHILE + IF
cambiarEstado(id, e)	Localiza el nodo y cambia su campo estado	WHILE + IF
eliminarProceso(id)	Redirige punteros y libera el nodo	IF-ELSE (cabeza) + WHILE (resto)
mostrarTodos()	Imprime todos los nodos en pantalla	WHILE (recorre toda la lista)

5.2.2 Cola de prioridad dinámica

La cola de prioridad dinámica se utilizará para simular la planificación del CPU. En esta estructura, los procesos no se ejecutan necesariamente en el orden en que ingresan, sino de acuerdo con su prioridad. Esto permite representar mejor el comportamiento de un sistema operativo, donde procesos importantes pueden ejecutarse antes que procesos secundarios.

Prioridad	Nivel	Ejemplo
1	Alta	Windows Defender, Actualización de Windows
2	Media	Visual Studio Code, Zoom, Google Meet.
3	Baja	Spotify, Descargas en segundo plano.

Operación	Acción	Estructura de control usada
encolar(proceso)	Inserta en la posición correcta según prioridad	IF-ELSE (frente vacío o mayor prioridad) + WHILE (busca posición)
desencolar()	Extrae y retorna el proceso del frente	IF (cola vacía)
verCola()	Muestra todos los procesos en espera	WHILE (recorre de frente a final)

5.2.3 Pila dinámica

La pila dinámica se utilizará para simular la asignación y liberación de memoria RAM. Esta estructura trabaja bajo el principio LIFO, es decir, el último elemento en entrar es el primero en salir. En el sistema, cuando un proceso inicia se registra su consumo de memoria, y cuando finaliza se libera el último bloque asociado.

Operación	Acción	Estructura de control usada
push(proceso, mb)	Apila un nuevo bloque de memoria en el tope	Asignación directa — sin bucle ($O(1)$)
pop()	Desapila y libera el bloque del tope	IF (pila vacía)
verEstado()	Muestra todos los bloques activos	WHILE (recorre de tope hacia abajo)

5.3. Algoritmos Principales

A continuación se presenta el pseudocódigo de los dos algoritmos centrales del sistema: agregar un proceso y cambiar el estado de un proceso. En cada uno se indica la estructura de programación empleada y su propósito.

5.3.1 Algoritmo 1: Agregar Proceso a la Lista Enlazada

Algoritmo AgregarProceso
Inicio

Solicitar ID del proceso
Solicitar nombre del proceso
Solicitar prioridad del proceso
Solicitar estado inicial
Solicitar memoria utilizada

Crear nuevo nodo de proceso
Asignar los datos ingresados al nodo

Si la lista está vacía Entonces
 El nuevo nodo será el inicio de la lista
Sino
 Recorrer la lista hasta el último nodo
 Enlazar el último nodo con el nuevo nodo
FinSi
Insertar el proceso en la cola de prioridad
Registrar memoria utilizada en la pila
Mostrar "Proceso agregado correctamente"

Fin

Explicación: este algoritmo registra un proceso nuevo, crea su nodo dinámico, lo inserta en la lista enlazada, lo organiza en la cola de prioridad y registra su uso de memoria.

5.3.2 Algoritmo 2: Cambiar Estado de un Proceso

Algoritmo CambiarEstadoProceso

Inicio

Solicitar ID del proceso
Buscar el proceso en la lista enlazada

Si el proceso existe Entonces
 Mostrar estado actual
 Solicitar nuevo estado
 Actualizar estado del proceso

Si el nuevo estado es "Finalizado" Entonces
 Liberar memoria asociada
 Retirar proceso de la cola de prioridad
FinSi

Mostrar "Estado actualizado correctamente"
Sino
 Mostrar "Proceso no encontrado"
FinSi

Fin

Explicación: este algoritmo busca un proceso y actualiza su estado. Si el proceso finaliza, el sistema libera la memoria y lo retira de la planificación.

5.3.3 Algoritmo 3: Ejecutar proceso según prioridad

Algoritmo EjecutarProcesoPrioridad

Inicio

Verificar si la cola de prioridad está vacía

Si la cola está vacía Entonces

Mostrar "No hay procesos pendientes"

Sino

Tomar el proceso ubicado al frente de la cola

Cambiar su estado a "En ejecución"

Mostrar datos del proceso ejecutado

Retirar el proceso de la cola

Cambiar estado a "Finalizado"

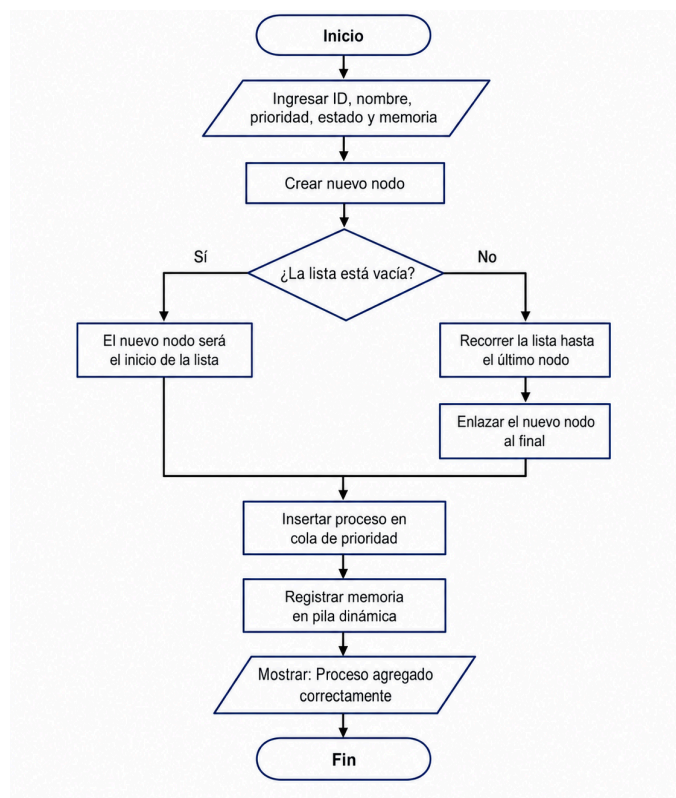
Liberar memoria utilizada por el proceso

FinSi

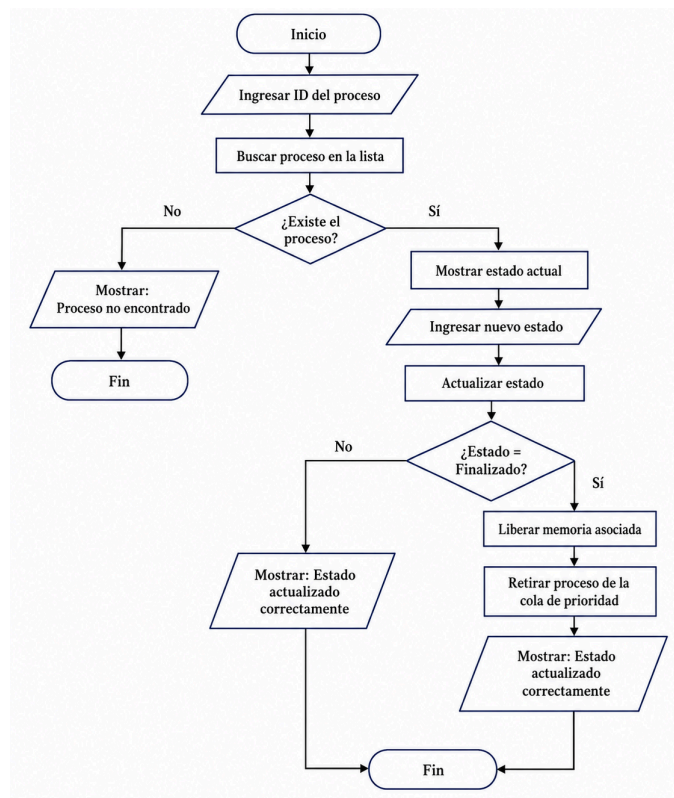
Fin

5.4 Diagramas de Flujo

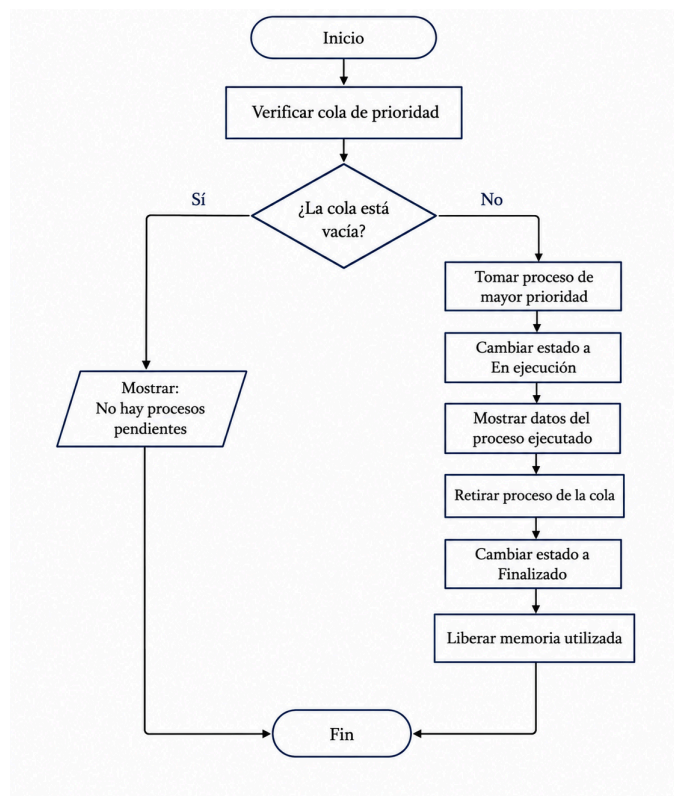
5.4.1 Diagrama de flujo para agregar proceso



5.4.2 Diagrama de flujo para cambiar estado del proceso



5.4.3 Diagrama de flujo para ejecutar proceso por prioridad



5.5 Justificación del Diseño

En esta sección se explica por qué cada estructura de datos es la más adecuada para su módulo correspondiente, analizando sus ventajas, limitaciones conocidas y eficiencia respecto a otras alternativas.

5.5.1 Comparación con alternativas

Módulo	Alternativa descartada	Estructura elegida	Razón del cambio
Gestor de procesos	Arreglo estático (array)	Lista enlazada simple	El arreglo requiere tamaño fijo. Los procesos se crean y destruyen dinámicamente, por lo que un array desperdiciaría memoria o se quedaría sin espacio.
Planificador CPU	Cola simple (FIFO puro)	Cola de prioridad dinámica	Una cola FIFO ejecutaría los procesos en orden de llegada, ignorando su urgencia. Windows Defender no puede esperar en fila detrás de Spotify.
Gestor de memoria	Lista enlazada invertida	Pila dinámica (LIFO)	El principio LIFO modela con precisión el comportamiento del stack de memoria de un proceso: el último bloque asignado es el primero en liberarse al cerrar.

5.5.2 Ventajas de la Lista Enlazada

- Tamaño dinámico: crece y decrece sin desperdiciar memoria. No hay que declarar un tamaño máximo de antemano.
- Inserción y eliminación eficiente: no requiere desplazar elementos como en un arreglo; basta con redirigir punteros.
- Refleja la realidad del sistema operativo: Windows mantiene su tabla de procesos como una estructura enlazada que se modifica continuamente.
- Facilita el recorrido completo: el WHILE permite listar todos los procesos activos en cualquier momento, mostrando el estado actualizado del sistema.

5.5.3 Ventajas de la Cola de Prioridad Dinámica

- Garantiza justicia por urgencia: los procesos críticos (Windows Defender, videollamadas) siempre se ejecutan antes que los de baja prioridad.

- Inserción ordenada automática: cada nuevo proceso se coloca en la posición correcta al encolarse, sin necesidad de reordenar la cola después.
- Simulación realista del planificador: reproduce el comportamiento de algoritmos como Priority Scheduling de Windows.
- Sin límite fijo de capacidad: al ser dinámica, acepta tantos procesos como la RAM del sistema permita.

5.5.4 Ventajas de la Pila Dinámica

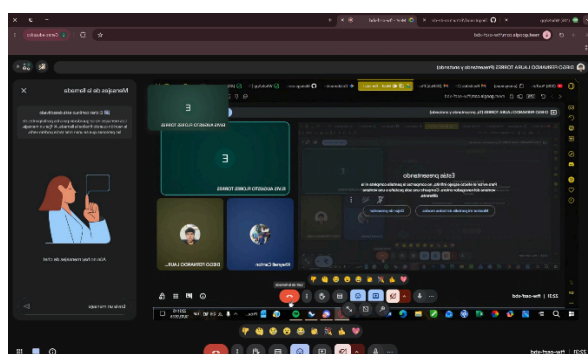
- LIFO apropiado para memoria: refleja cómo el sistema operativo libera el stack de un proceso: la última función llamada es la primera en retornar y liberar su memoria.
- Push y pop en tiempo constante: ambas operaciones actúan sobre el tope sin recorrer la estructura, sin importar cuántos elementos haya apilados.
- Historial natural de procesos cerrados: al registrar cada proceso terminado en la pila, el historial muestra automáticamente el último proceso cerrado en primer lugar.
- Implementación sencilla y robusta: al operar siempre sobre el tope, el único error posible es el pop sobre una pila vacía, fácilmente controlado con un IF.

5.5.6 Eficiencia General del Sistema

El diseño combina las tres estructuras de manera que cada una atiende una necesidad distinta sin solaparse, lo que permite que el sistema sea modular y fácil de mantener:

Operación	Estructuras de control	Por qué es eficiente
Agregar proceso	WHILE + IF-ELSE	Un solo recorrido; sin reorganización de datos.
Buscar / modificar / cambiar estado	WHILE + IF	Se detiene en cuanto encuentra el proceso; no recorre el resto.
Eliminar proceso	IF-ELSE + WHILE	Solo redirige dos punteros; no mueve ningún dato.
Ejecutar proceso (desencolar)	IF	Extrae siempre el frente; operación inmediata.
Asignar / liberar memoria (push/pop)	IF (solo en pop)	Actúa sobre el tope; no requiere recorrido.

6. Evidencias de trabajo Grupal:



ElvisFloresT

Collaborator

HunterHive1233

Collaborator

Contributors 1

DiegoLauraT

100% como abrir en

Recibidos (280) - 0

GitHub! Please ver

Meet - the

Colaboración en G

estructura-de-d

DiegoLauraT / Estructura-de-dator-ABR---K-DOG-URBAN 0

Type to search

Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

Settings

Files

main

Go to file

gitignore

Guia Completa_Capitulo1_KDogUrban.pdf

README.md

Estructura-de-dator-ABR---K-DOG-URBAN / Guia Completa_Capitulo1_KDogUrban.pdf

DiegoLauraT Add files via upload 67a00a · 7 minutes ago

7:79:08

History

22:23 | ftw-oszf-xbd

...

📄

🗨

📺

📧

👤

⋮

📞

Mensajes de la llamada

El chat continuo está desactivado

Los mensajes no se guardarán para los participantes de la reunión cuando finalice la llamada. Al fijar un mensaje, las personas que se unan más tarde podrán verlo.

Aún no hay mensajes de chat

Envía un mensaje

E

ELVIS AUGUSTO FLORES TORRES

Kheynell Carrion

DIEGO FERNANDO LAURA TORRES

📄

🗨

📺

📧

👤

⋮

📞

16:26 | ozo-xauz-ggz

Historial de versiones

Todas las versiones ▼

> 20 de mayo, 17:04



Versión actual

- DIEGO FERNANDO LAURA TORRES
- ELVIS AUGUSTO FLORES TORRES
- KHEYNELL ALESSANDER CARRIÓN
HERQUINIO

6.1 Diapositiva – Canva:

<https://canva.link/dz7mkbxj2k54gt>

6.2 Enlace GITHUB

<https://github.com/DiegoLauraT/ESTRUCTURA-DE-DATOS---EVALUACION---EQUIPO-5.git>